

Cyberboss 优化计划书 v0.2.0

基于当前 v0.1.0 架构，规划下一阶段版本管理策略与四项新能力的完整实施方案。

目录

- 1. 版本管理策略
- 2. 碎片化记忆（通勤学习）
- 3. 每日总结
- 4. 闪存记忆
- 5. 大构思完善
- 6. 实施路线图
- 7. 工具设计参考

1. 版本管理策略

1.1 现状分析

项目	当前状态
版本号	0.1.0 (package.json)
仓库	GitHub WenXiaoWendy/cyberboss
协议	AGPL-3.0-only
Git	未在本本地初始化（当前工作目录非 git repo）
发布方式	无 npm 发布，用户通过 git clone + npm install 使用
依赖管理	npm + GitHub 直接依赖 (timeline-for-agent, whereabouts-mcp)

1.2 版本号规范：语义化版本 (SemVer)

MAJOR.MINOR.PATCH

段	触发条件	示例
MAJOR	架构重构、不兼容的 API 变更、runtime 层大改	0.x → 1.0.0 正式发布
MINOR	新增能力模块、新增 MCP 工具、新增微信命令	0.1.0 → 0.2.0（本计划）
PATCH	Bug 修复、性能优化、文档更新、小调整	0.2.0 → 0.2.1

1.3 分支策略

```
main      ← 稳定版本, tag 对齐 SemVer
└─ dev    ← 日常开发集成分支
    ├── feature/fragmented-memory ← 碎片化记忆
    ├── feature/daily-summary     ← 每日总结
    ├── feature/flash-memory      ← 闪存记忆
    └─ feature/idea-refinement    ← 大构思完善
```

规则:

- **main** 只接受 PR merge, 每次 merge 打 tag (**v0.2.0**, **v0.2.1**...)
- **dev** 是所有 feature 分支的集成目标
- feature 分支命名: **feature/<slug>** 或 **fix/<slug>**
- 不直接在 **main** 上 commit

1.4 发布流程

```
feature 开发 → PR → dev 集成 → 手动测试 → PR → main → git tag vX.Y.Z
```

因项目不发布 npm, tag 即发布。CHANGELOG.md 随 tag 更新。

1.5 Changelog 规范

采用 [Keep a Changelog](#) 格式, 分类:

- **Added** — 新能力
- **Changed** — 行为变更
- **Deprecated** — 即将移除
- **Removed** — 已移除
- **Fixed** — 修复
- **Security** — 安全修复

1.6 Git 初始化 (待执行)

```
cd D:\Cyberboss\cyberboss
git init
git add .
git commit -m "chore: initial commit v0.1.0"

Co-Authored-By: Claude <noreply@anthropic.com>
git tag v0.1.0
git remote add origin https://github.com/WenXiaoWendy/cyberboss.git
```

难度: ★☆☆☆☆ | 工作量: 小 (一次性)

2. 碎片化记忆 (通勤学习)

2.1 功能描述

用户通过微信告知"在坐车"、"在通勤"等状态后，Cyberboss 自动从本地知识库中随机抽取内容，以**问答**或**冷知识讲解**的形式进行互动，把碎片时间变成学习时间。

2.2 交互流程

```
用户: "在通勤了, 大概20分钟"
↓
Cyberboss: 识别通勤状态 + 估算时间窗口
↓
从知识库随机抽题/冷知识
↓
Cyberboss: "好, 来一题: Python 的 GIL 是什么?"
用户: "全局解释器锁"
↓
Cyberboss: 判定正误 → 给出简短解释 → 下一题
↓
(时间耗尽或用户喊停)
Cyberboss: "到了吧? 今天通勤刷了 3 题, 正确率 2/3。"
```

2.3 数据结构设计

知识库目录结构

```
${HOME}/.cyberboss/knowledge-base/
├── index.json           ← 全局索引
├── computer-science/   ← 分类目录
│   ├── _meta.json      ← 分类元信息
│   ├── python-gil.json ← 单个知识点
│   └── tcp-vs-udp.json
├── history/
│   ├── _meta.json
│   └── three-kingdoms-01.json
├── science/
│   ├── _meta.json
│   └── quantum-entanglement.json
└── life-hacks/
    ├── _meta.json
    └── coffee-metabolism.json
```

单个知识点 JSON Schema

```
{
  "id": "cs-python-gil-001",
  "type": "quiz",
  "category": "computer-science",
```

```

"tags": ["python", "concurrency", "intermediate"],
"difficulty": "medium",
"estimatedSeconds": 120,
"question": "Python 的 GIL（全局解释器锁）的全称是什么？它主要影响什么场景？",
"answer": {
  "keywords": ["global interpreter lock", "全局解释器锁", "多线程", "CPU密集"],
  "explanation": "GIL 是 CPython 的互斥锁，保证同一时刻只有一个线程执行 Python 字节码。它主要限制 CPU 密集型多线程任务的并行性能，但对 I/O 密集型任务影响较小。"
},
"source": "Python官方文档",
"createdAt": "2026-06-18T10:00:00+08:00",
"lastReviewedAt": null,
"reviewCount": 0,
"correctCount": 0
}

```

冷知识条目 Schema

```

{
  "id": "science-quantum-001",
  "type": "cold-fact",
  "category": "science",
  "tags": ["physics", "quantum", "mind-blowing"],
  "estimatedSeconds": 60,
  "title": "量子纠缠不是超光速通信",
  "content": "量子纠缠确实可以瞬时\"关联\"两个粒子状态，但你无法利用它来传递信息—因为测量结果是随机的，你无法控制对方看到的是什么。爱因斯坦称之为\"鬼魅般的超距作用\"，但信息传递仍然受光速限制。",
  "source": "Quantum Mechanics: The Theoretical Minimum",
  "createdAt": "2026-06-18T10:00:00+08:00"
}

```

全局索引 index.json

```

{
  "version": 1,
  "categories": {
    "computer-science": { "label": "计算机科学", "count": 42, "icon": "💻" },
    "history": { "label": "历史", "count": 15, "icon": "📖" },
    "science": { "label": "科学", "count": 28, "icon": "🔬" },
    "life-hacks": { "label": "生活技巧", "count": 10, "icon": "💡" }
  },
  "sessionState": {
    "lastCategoryUsed": "computer-science",
    "recentItemIds": ["cs-python-gil-001", "cs-tcp-udp-002"],
    "recentWindowSize": 20
  },
  "stats": {

```

```
"totalItems": 95,  
"totalReviews": 342,  
"overallCorrectRate": 0.73  
}  
}
```

2.4 核心逻辑

1. 触发检测：消息中包含 [通勤，坐车，地铁，公交，路上，等车，碎片]
2. 时间估算：
 - 用户明确说 "20分钟" → 使用 20min
 - 用户说"一会" → 默认 15min
 - 未指定 → 默认无限，直到用户喊停
3. 选题策略（加权随机）：
 - 从未答过的知识点优先（权重 ×2）
 - 答错过的知识点适度回顾（权重 ×1.3）
 - 最近已答过的排除（20 条窗口）
 - 难度匹配剩余时间
4. 交互节奏：
 - 出题 → 等待回答 → 判断 → 简短解释 → 停顿3s → 下一题
 - 冷知识：直接讲解 → 停顿5s → 下一则
5. 结束条件：
 - 用户喊停（"到了""不做了""停"）
 - 估算时间耗尽
6. 总结输出：本次答了几题、正确率、学习时长

2.5 工具/服务接口设计

新增 MCP Tool: cyberboss_knowledge_quiz

```
// 随机获取一道题  
cyberboss_knowledge_quiz({  
  action: "next",  
  category?: "computer-science", // 可选：指定分类  
  difficulty?: "easy" | "medium" | "hard",  
  excludeIds?: string[] // 排除已答过的  
}) => QuizItem | ColdFactItem | null  
  
// 提交答案  
cyberboss_knowledge_quiz({  
  action: "submit",  
  itemId: "cs-python-gil-001",  
  userAnswer: "全局解释器锁"  
}) => { correct: boolean, explanation: string }  
  
// 获取知识库状态  
cyberboss_knowledge_quiz({
```

```
    action: "status"
  }) => { totalItems, categories, recentStats }
```

新增 MCP Tool: cyberboss_knowledge_manage

```
// 添加知识点
cyberboss_knowledge_manage({
  action: "add",
  item: QuizItem | ColdFactItem
}) => { id: string }

// 导入批量知识
cyberboss_knowledge_manage({
  action: "import",
  filePath: "/path/to/batch-import.json"
}) => { imported: number, skipped: number }
```

2.6 服务端文件

```
src/services/knowledge-service.js    ← 知识库读写、索引维护、选题策略
src/services/knowledge-quiz-session.js ← 单次通勤会话状态管理
```

2.7 难度评估

维度	评级	说明
数据结构设计	★★☆☆☆	JSON Schema 已定义，直接实现
选题算法	★★☆☆☆	加权随机 + 去重窗口，逻辑清晰
会话管理	★★★★☆	需要维护通勤会话状态、计时
微信交互	★★☆☆☆	复用现有消息通道，无需新增
知识库内容	★★★★★	核心难点：初始内容从哪来
综合	★★★★☆	技术难度中等，内容建设是瓶颈

3. 每日总结

3.1 功能描述

每天定时（或用户主动触发）生成一份结构化的日终报告，聚合并总结当日的：

- Timeline 事件
- Diary 碎片
- 闪存记忆（见第 4 节）

- 通勤学习记录
- 工作完成情况

3.2 交互流程

方案A – 定时触发：

21:00 → Cyberboss 自动生成总结 → 微信推送摘要 → 可选展开详情

方案B – 完成触发：

用户："今天收工了"

Cyberboss："收到，正在汇总你今天的数据..."

→ 推送完整日终总结

方案C – 混合模式（推荐）：

- 定时 21:00 生成基础草稿
- 用户收工时确认 + 补充
- 最终版本存入日记

3.3 总结模板

📅 2026-06-18 日终总结

🕒 时间轨迹

- 09:30 - 12:00 Coding (2h30m)
- 12:00 - 12:40 午餐
- 12:40 - 14:00 午休 / 碎片刷题
- 14:00 - 17:50 项目开发 (3h50m)
- 17:50 - 18:20 通勤（刷了3题）

💡 今日灵感

- "要不要给 Cyberboss 加一个喝水提醒功能？" (14:23)
- "周末想去试一下那家新开的拉面店" (17:15)

✅ 完成事项

- [x] 修复 Windows IPC TCP 问题
- [x] 完成 v0.2.0 计划书初稿

📖 日记片段

- 今天状态还行，下午比较集中
- 中午又忘记吃维生素了...

💬 学习记录

- 通勤刷题 3 题，正确 2/3
- 正确率：67%

🌙 明天计划

- （用户补充或 AI 根据未完成事项建议）

 由 Cyberboss v0.2.0 自动生成 • 2026-06-18 21:00

3.4 数据聚合逻辑

```
DailySummaryAggregator
├─ TimelineAggregator    ← 读取 timeline 数据, 统计时间段
├─ DiaryAggregator       ← 读取 diary/ 当日条目
├─ FlashMemoryAggregator ← 读取闪存记忆当日条目
├─ QuizAggregator        ← 读取知识库通勤记录
└─ TaskAggregator        ← 读取 reminder/task 完成状态
```

3.5 工具/服务接口

新增 MCP Tool: `cyberboss_daily_summary`

```
// 生成日终总结
cyberboss_daily_summary({
  action: "generate",
  date?: "2026-06-18",          // 默认今天
  format?: "full" | "brief",    // 完整版 vs 摘要版
  includeSections?: ["timeline", "flash", "diary", "quiz", "tasks"]
}) => { summary: string, sections: {...} }

// 获取总结状态
cyberboss_daily_summary({
  action: "status"
}) => { lastGenerated, todayDraftExists, sectionsAvailable }

// 追加明天计划
cyberboss_daily_summary({
  action: "append_plan",
  date: "2026-06-18",
  plan: "明天要完成 API 文档"
}) => { ok: true }
```

3.6 定时调度

复用现有 `system-checkin-poller.js` 的随机轮询机制, 新增一个固定时间检查点:

```
// src/services/daily-summary-scheduler.js
// 每天 21:00 触发一次 summary checkin
// 如果用户当天有活动 → 生成总结并推送
// 如果用户当天无活动 → 跳过
```


3.7 难度评估

维度	评级	说明
数据聚合	★★★★☆☆	需要跨 4-5 个数据源 join
模板生成	★★☆☆☆☆	Markdown 模板，逻辑简单
定时调度	★★☆☆☆☆	复用 checkin 机制，加固定时间点
完成检测	★★☆☆☆☆	"收工了"等关键词匹配
总结质量	★★★★☆☆	需要好的 prompt 驱动模型生成
综合	★★★★☆☆	技术难度中等，主要工作在聚合层

4. 闪存记忆

4.1 功能描述

用户在微信里随口说的灵感、想法、待办念头，Cyberboss 自动识别并存入专门的"闪存记忆"仓库。定期对积累的闪存进行整理、去重、归类，并在合适时机提醒用户回顾。

4.2 交互流程

用户："话说我突然想到，是不是可以写个自动整理书签的脚本"

Cyberboss：（自动识别为灵感/想法）

- 存入闪存记忆
- 回复："💡 已存。标签：工具脚本、自动化"

--- 几天后的 checkin ---

Cyberboss："你这周攒了 7 条闪存灵感，要不要现在快速过一遍？"

用户："好"

Cyberboss：

- "1. 💡 自动整理书签脚本 - 工具脚本
- 2. 💡 周末去试新拉面店 - 生活
- 3. 💡 给项目加 CI/CD - 开发
- ...

你想深入聊聊哪个？还是归档一些？"

4.3 数据结构

闪存条目存储

```
${HOME}/.cyberboss/flash-memory/  
├─ index.json      ← 全局索引和状态  
├─ inbox/          ← 待整理
```

```

├── fm_20260618_001.json
├── categorized/      ← 已分类
│   ├── dev/
│   ├── life/
│   ├── idea/
│   └── todo/
├── archived/        ← 已归档（不再活跃）
└── merged/          ← 已合并到大构思

```

单条闪存 JSON

```

{
  "id": "fm_20260618_001",
  "sourceType": "wechat",
  "rawText": "话说我突然想到，是不是可以写个自动整理书签的脚本",
  "cleanedText": "写一个自动整理书签的脚本",
  "category": "idea",
  "tags": ["工具脚本", "自动化", "浏览器"],
  "status": "inbox",
  "priority": "low",
  "relatedFlashIds": [],
  "mergedToIdeaId": null,
  "createdAt": "2026-06-18T14:23:00+08:00",
  "reviewedAt": null,
  "archivedAt": null
}

```

索引 index.json

```

{
  "version": 1,
  "counts": {
    "inbox": 5,
    "categorized": 23,
    "archived": 10,
    "total": 38
  },
  "categories": ["dev", "life", "idea", "todo"],
  "lastReviewAt": "2026-06-15T20:30:00+08:00",
  "reviewIntervalDays": 3
}

```

4.4 智能识别策略

1. 触发模式匹配（在模型层做，不在规则层）：
 - "我突然想到" "话说" "要不要" "是不是可以" → 灵感

- "得记得" "别忘了" "记得要" → 待办
 - "好想去" "想试试" → 生活愿望
 - "这个bug是因为" "问题出在" → 技术备忘
2. 不触发的场景:
- 直接的命令/问答
 - `timeline/diary` 操作
 - 已经明确的 `checkin` 回复
3. 置信度机制:
- 模型判断是否为"值得存的念头脑暴"
 - 低置信度时不存, 避免噪音

4.5 工具/服务接口

新增 MCP Tool: `cyberboss_flash_memory`

```
// 存入闪存
cyberboss_flash_memory({
  action: "capture",
  text: "写一个自动整理书签的脚本",
  category?: "idea",
  tags?: ["工具脚本", "自动化"]
}) => { id: "fm_20260618_001" }

// 批量列出
cyberboss_flash_memory({
  action: "list",
  status?: "inbox" | "categorized" | "all",
  category?: "idea",
  limit?: 10,
  offset?: 0
}) => { items: FlashItem[], total: number }

// 整理单条
cyberboss_flash_memory({
  action: "update",
  id: "fm_20260618_001",
  updates: { category: "dev", tags: ["..."], status: "categorized" }
}) => { ok: true }

// 批量整理（合并、归档、关联）
cyberboss_flash_memory({
  action: "batch_update",
  operations: [
    { action: "categorize", ids: ["001", "002"], category: "dev" },
    { action: "merge", sourceIds: ["003", "004"], into: "005" },
    { action: "archive", ids: ["006"] }
  ]
}) => { results: [...] }
```

```
// 获取回顾建议
cyberboss_flash_memory({
  action: "review_suggestions",
  since?: "2026-06-15"
}) => { inboxCount: 5, suggestedItems: [...], lastReviewAt: "..." }
```

4.6 定期整理逻辑

```
checkin 触发 → 检测到空闲状态 →
├─ inbox 积压 > 5 条 → 提醒用户整理
├─ 距上次回顾 > 3 天 → 提醒回顾
└─ 检测到可合并条目 → 建议合并
```

4.7 难度评估

维度	评级	说明
存储层	★★☆☆☆	JSON 文件存储，类比 diary
识别触发	★★★★☆	依赖模型判断，需要好的 prompt
分类/标签	★★★★☆	自动标签需要一定 NLP 能力
整理逻辑	★★★★☆	去重、合并、关联需要算法设计
回顾交互	★★☆☆☆	列表展示 + 用户选择，不复杂
综合	★★★★☆	中等难度，关键是识别准确率

5. 大构思完善

5.1 功能描述

用户在电脑端（项目工作区）放置一个"大构思"草稿文件，Cyberboss 在检测到用户空闲时，主动通过微信发起对话，用提问的方式帮助用户完善构思——类似一个耐心的产品经理/架构师陪你做头脑风暴。

5.2 交互流程

```
电脑端：
  用户在 D:\Cyberboss\ideas\ 下放了 draft-my-saas-app.md
  （或放在 ~/.cyberboss/ideas/drafts/ 下）
  ↓
Cyberboss 检测到新草稿 + 用户当前空闲
  ↓
微信端：
  Cyberboss: "我看到你放了一个新项目的构思「我的 SaaS 应用」，
              现在有空聊聊吗？我想帮你把这个想法捋清楚。"
  用户: "好"
```

Cyberboss: "首先, 这个 SaaS 解决的是谁的什么问题?"
用户: "独立开发者, 他们不知道怎么定价"
Cyberboss: "明白了。那你觉得现在的定价工具最大的问题是什么?"
...

(经过 5-10 轮对话)

Cyberboss: "我已经把你的回答整理成了一份新的草稿, 更新在你的构思文件夹里。
核心假设还需要验证的标了 ⚠️, 想继续的时候告诉我。"

5.3 目录结构

<code>\${CYBERBOSS_WORKSPACE_ROOT}/ideas/</code>	← 项目级构思 (或)
<code>\${HOME}/.cyberboss/ideas/</code>	← 全局构思目录
├─ drafts/	← 用户的原始构思草稿
│ └─ my-saas-app.md	
├─ refined/	← AI 完善后的版本
│ └─ my-saas-app_v2.md	
├─ sessions/	← 完善对话的会话记录
│ └─ my-saas-app_session_01.json	
└─ index.json	← 构思索引

5.4 草稿文件约定

用户在草稿里用 frontmatter 标记状态和关注点:

```
---
status: draft
focus_areas: [target_user, monetization, tech_stack]
created: 2026-06-18
---

# 我的 SaaS 应用

一个帮助独立开发者定价的工具。

## 初步想法
- 输入产品特性 → 输出建议定价区间
- 竞品分析集成
- ...

## 我的困惑
- 不知道目标用户画像是啥
- 不确定怎么收费 (月付 vs 一次性)
```

Cyberboss 解析 `focus_areas` 和 `我的困惑`, 以此为起点生成提问策略。

5.5 提问策略引擎

框架：Socratic Questioning + 产品思维

Phase 1: 澄清问题 (Clarify)

- "这个想法解决的是什么问题？"
- "谁会因为这个方案受益？"

Phase 2: 挑战假设 (Challenge)

- "如果用户不愿意付费，还有什么其他价值？"
- "有没有更简单的方式达到同样目的？"

Phase 3: 探索视角 (Perspective)

- "如果从竞品的角度看，他们会怎么回应？"
- "半年后回头看，这个决策最关键的是什么？"

Phase 4: 落地推演 (Implement)

- "第一版最少需要哪些功能才能验证假设？"
- "如果要两周内上线 MVP，你会砍掉什么？"

Phase 5: 总结整合 (Synthesize)

- 生成完善版草稿
- 标注未解决问题
- 建议下一步行动

提问选择算法

1. 读取草稿 `frontmatter` → 确定 `focus_areas`
2. 根据 `focus_areas` 从题库中抽取初始问题
3. 每轮对话后评估：
 - 该领域还有未覆盖的角度吗？
 - 用户的回答是否揭示了新的关注点？
4. 动态调整下一轮问题
5. 覆盖率达标或用户喊停 → 进入 Phase 5

5.6 空闲检测机制

复用现有 `checkin` 轮询 + 新增空闲判定：

- ```
// 空闲判定条件（全部满足）：
```
1. 最近一次用户消息距今 > 15 分钟
  2. 当前时间在工作时间窗口内（如 09:00-22:00）
  3. 没有进行中的通勤学习会话
  4. 没有未回复的 `reminder`
  5. `ideas/drafts/` 中有待完善的草稿（`status: draft`）

## 5.7 工具/服务接口

## 新增 MCP Tool: cyberboss\_idea\_refinement

```
// 列出所有构思草稿
cyberboss_idea_refinement({
 action: "list_drafts"
}) => { drafts: IdeaDraft[], total: number }

// 读取草稿内容
cyberboss_idea_refinement({
 action: "read_draft",
 draftId: "my-saas-app"
}) => { content: string, frontmatter: {...}, lastRefinedAt: string | null }

// 写入完善后的版本
cyberboss_idea_refinement({
 action: "write_refined",
 draftId: "my-saas-app",
 content: "...", // 完善后的 markdown
}) => { version: "v2", path: "/path/to/refined/my-saas-app_v2.md" }

// 记录完善会话
cyberboss_idea_refinement({
 action: "log_session",
 draftId: "my-saas-app",
 session: { turns: [...] }
}) => { sessionId: "..." }

// 获取完善建议（提问列表）
cyberboss_idea_refinement({
 action: "suggest_questions",
 draftId: "my-saas-app",
 phase?: "clarify" | "challenge" | "perspective" | "implement"
}) => { questions: string[], currentPhase: string }

// 更新草稿状态
cyberboss_idea_refinement({
 action: "update_status",
 draftId: "my-saas-app",
 status: "draft" | "refining" | "refined" | "archived"
}) => { ok: true }
```

## 5.8 服务端文件

|                                         |             |
|-----------------------------------------|-------------|
| src/services/flash-memory-service.js    | ← 闪存记忆 CRUD |
| src/services/knowledge-service.js       | ← 知识库管理     |
| src/services/knowledge-quiz-session.js  | ← 通勤答题会话    |
| src/services/daily-summary-service.js   | ← 每日总结聚合    |
| src/services/daily-summary-scheduler.js | ← 总结定时器     |

src/services/idea-refinement-service.js ← 大构思完善

src/services/idea-refinement-session.js ← 完善会话管理

5.9 难度评估

| 维度   | 评级     | 说明                           |
|------|--------|------------------------------|
| 草稿解析 | ★★☆☆☆  | frontmatter + markdown，简单    |
| 空闲检测 | ★★★★☆☆ | 需要多个信号融合判定                   |
| 提问引擎 | ★★★★☆  | <b>核心难点：</b> 需要高质量的领域问题和动态调整 |
| 对话管理 | ★★★★☆  | 多轮对话的上下文维护和总结                |
| 完善输出 | ★★★★☆☆ | 基于对话内容生成结构化文档                |
| 综合   | ★★★★☆  | 技术难度最高的一项，提问质量和对话体验是关键       |

6. 实施路线图

6.1 总体排期

v0.1.0 (当前)

Phase 0: 基础设施 (预计 1-2 天)

0.1 Git 初始化 + 分支策略落地

0.2 CHANGELOG.md 创建

0.3 工具注册框架扩展 (支持新 MCP tools 热注册)

Phase 1: 闪存记忆 (预计 3-5 天) ★★★★★

1.1 flash-memory 数据模型 + 存储层

1.2 cyberboss\_flash\_memory MCP Tool

1.3 微信消息 → 闪存自动识别

1.4 定期整理提醒

Phase 2: 碎片化记忆 (预计 4-6 天) ★★★★★

2.1 knowledge-base 数据结构 + 初始内容导入

2.2 cyberboss\_knowledge\_quiz MCP Tool

2.3 通勤会话管理 (计时、选题、判题)

2.4 会话统计与总结

Phase 3: 每日总结 (预计 3-4 天) ★★★★★

3.1 数据聚合层 (timeline + diary + flash + quiz)

3.2 cyberboss\_daily\_summary MCP Tool

3.3 定时调度 (21:00 固定 checkin)

3.4 总结模板与微信推送

Phase 4: 大构思完善 (预计 5-8 天) ★★★★★

4.1 草稿目录 + 文件解析

4.2 cyberboss\_idea\_refinement MCP Tool

4.3 提问引擎 (Socratic 五阶段)



- 4.4 空闲检测 + 主动发起
  - 4.5 完善版草稿生成与回写
- v0.2.0 发布
  - CHANGELOG 汇总
  - 文档更新 (README、commands、architecture)
  - git tag v0.2.0

6.2 依赖关系



闪存记忆是**基础模块**——每日总结和构思完善都会引用闪存数据。

6.3 风险与缓解

| 风险                       | 概率 | 影响              | 缓解措施                                    |
|--------------------------|----|-----------------|-----------------------------------------|
| 知识库内容不足                  | 高  | 碎片化记忆功能空转       | Phase 2 前先准备 50+ 条种子内容                  |
| 闪存识别误判率高                 | 中  | 噪音过多，用户反感       | 保守策略：不确定时不存；提供 <code>/unflash</code> 撤回 |
| 提问引擎质量差                  | 中  | 用户体验不佳          | 先用固定题库 + 人工审核，迭代优化                      |
| Claude Code context 消耗过大 | 中  | 多能力并行时 token 不够 | 每个能力独立 thread 或 compact 策略              |
| 用户空闲检测不准                 | 低  | 在不合适的时机打扰       | 多层信号融合，宁可漏判不要误判                         |

7. 工具设计参考

7.1 现有工具清单 (v0.1.0)

|                              |           |
|------------------------------|-----------|
| MCP Tools (cyberboss_tools): |           |
| cyberboss_channel_send_file  | ← 发送文件到微信 |
| cyberboss_diary_append       | ← 追加日记    |
| cyberboss_reminder_create    | ← 创建提醒    |
| cyberboss_system_send        | ← 系统消息队列  |
| cyberboss_timeline_write     | ← 写入时间轴   |

|                                     |             |
|-------------------------------------|-------------|
| — cyberboss_timeline_build          | ← 构建时间轴站点   |
| — cyberboss_timeline_serve          | ← 启动时间轴静态服务 |
| — cyberboss_timeline_dev            | ← 启动时间轴开发服务 |
| — cyberboss_timeline_screenshot     | ← 时间轴截图     |
| — cyberboss_timeline_read           | ← 读取时间轴     |
| — cyberboss_timeline_categories     | ← 时间轴分类     |
| — cyberboss_timeline_proposals      | ← 时间轴建议     |
| — cyberboss_sticker_tags            | ← 表情包标签     |
| — cyberboss_sticker_pick            | ← 选取表情包     |
| — cyberboss_sticker_send            | ← 发送表情包     |
| — cyberboss_sticker_delete          | ← 删除表情包     |
| — cyberboss_sticker_save_from_inbox | ← 保存表情包     |
| — cyberboss_sticker_update          | ← 更新表情包     |
| — whereabouts_* × 5                 | ← 行踪服务      |

WeChat Commands:

|                                 |
|---------------------------------|
| — /bind, /status, /new, /reread |
| — /compact, /stop, /switch      |
| — /checkin, /chunk              |
| — /yes, /always, /no            |
| — /model, /star, /help          |

7.2 v0.2.0 新增工具设计

新增 MCP Tools:

|                              |                  |
|------------------------------|------------------|
| — cyberboss_knowledge_quiz   | ← 碎片化记忆：出题/判题/状态 |
| — cyberboss_knowledge_manage | ← 碎片化记忆：知识库管理    |
| — cyberboss_daily_summary    | ← 每日总结：生成/追加/状态  |
| — cyberboss_flash_memory     | ← 闪存记忆：捕获/列表/整理  |
| — cyberboss_idea_refinement  | ← 大构思完善：草稿/提问/回写 |

建议新增 WeChat Commands:

|                          |            |
|--------------------------|------------|
| — /quiz <category>       | ← 手动触发碎片刷题 |
| — /quiz stop             | ← 停止刷题     |
| — /summary               | ← 手动触发日终总结 |
| — /summary <date>        | ← 查看某日总结   |
| — /flash <text>          | ← 手动存入闪存   |
| — /flash list            | ← 列出闪存     |
| — /flash review          | ← 回顾闪存     |
| — /idea                  | ← 列出所有构思草稿 |
| — /idea refine <draftId> | ← 开始完善某个构思 |
| — /idea stop             | ← 停止完善会话   |

7.3 项目目录变更

|                       |      |
|-----------------------|------|
| src/                  |      |
| — services/           |      |
| — diary-service.js    | (已有) |
| — reminder-service.js | (已有) |

```

├─ sticker-service.js (已有)
├─ system-message-service.js (已有)
├─ channel-file-service.js (已有)
├─ timeline-service.js (已有)
├─ vision-context.js (已有)
├─ flash-memory-service.js ← 新增
├─ knowledge-service.js ← 新增
├─ knowledge-quiz-session.js ← 新增
├─ daily-summary-service.js ← 新增
├─ daily-summary-scheduler.js ← 新增
├─ idea-refinement-service.js ← 新增
├─ idea-refinement-session.js ← 新增
├─ tools/
│ └─ tool-host.js (已有, 需扩展注册)
│ └─ create-project-tooling.js (已有, 需扩展)
│ └─ mcp-stdio-server.js (已有)
│ └─ runtime-context-store.js (已有)
├─ ${HOME}/.cyberboss/
│ └─ knowledge-base/ ← 新增: 知识库
│ └─ flash-memory/ ← 新增: 闪存记忆仓库
│ └─ ideas/ ← 新增: 大构思目录 (全局)
│ └─ daily-summaries/ ← 新增: 日终总结存档
│ └─ diary/ (已有)
│ └─ stickers/ (已有)
│ └─ timeline/ (已有)
└─ ...

```

## 7.4 设计原则（对齐现有架构）

回顾 [docs/architecture.md](#) 的核心约定：

### 1. Capability 不耦合到 Core

- 新能力全部作为 `integrations/*` 或 `services/*`
- Core 只管配置读取和编排

### 2. 外部依赖尽可能独立项目

- 知识库内容建设可考虑独立 cli 工具
- 闪存记忆的 web 浏览面板可做独立 mini-site

### 3. Channel / Runtime 适配器不感知能力细节

- 微信消息识别闪存 → 在 runtime 层（模型判断），不在 channel 层
- WeChat Command 只做路由，不写业务逻辑

### 4. Project Tools 是模型与能力的唯一接口

- 所有新能力通过 MCP Tool 暴露
- 微信命令可触发 → 转化为 tool 调用 → 能力执行

## 附录 A：初始知识库种子内容建议

为了 Phase 2 不空转，建议首批准备 50-100 条知识点，覆盖：

| 分类      | 建议数量 | 来源建议       |
|---------|------|------------|
| 计算机科学   | 20 条 | 面试题、技术博客精华 |
| 历史冷知识   | 10 条 | 维基百科冷知识条目  |
| 科学常识    | 10 条 | 科普类书籍摘要    |
| 生活技巧    | 10 条 | 实用生活经验     |
| 经济学/心理学 | 10 条 | 行为经济学、认知偏差 |

格式化为上述 JSON Schema，放入对应分类目录即可。

## 附录 B：关键设计决策点（待确认）

| # | 决策       | 选项 A         | 选项 B                      | 建议                                    |
|---|----------|--------------|---------------------------|---------------------------------------|
| 1 | 每日总结触发方式 | 纯定时（21:00）   | 定时 + 手动混合                 | <b>B</b> ：更灵活                         |
| 2 | 闪存识别     | 全自动（每条消息都判断） | 关键词触发                     | <b>B</b> ：减少噪音                        |
| 3 | 构思完善启动   | 纯空闲检测自动发起    | 用户手动 / <i>idea refine</i> | <b>B 优先</b> ：先手动，稳定后加自动               |
| 4 | 知识库存储    | 本地 JSON 文件   | SQLite                    | <b>A</b> ：对齐现有设计（全部 JSON 文件）          |
| 5 | 通勤检测     | 关键词匹配        | 行踪服务自动判断（移动 + 时间）         | <b>B 长期</b> ：先 A 快速上线，后续接 whereabouts |

文档版本: v1.0 创建日期: 2026-06-18 作者: Claude (Cyberboss Agent) 下步行动: 小方确认方向后，进入 Phase 0（Git 初始化 + 工具注册框架扩展）